Java main() Method – public static void main(String[] args)

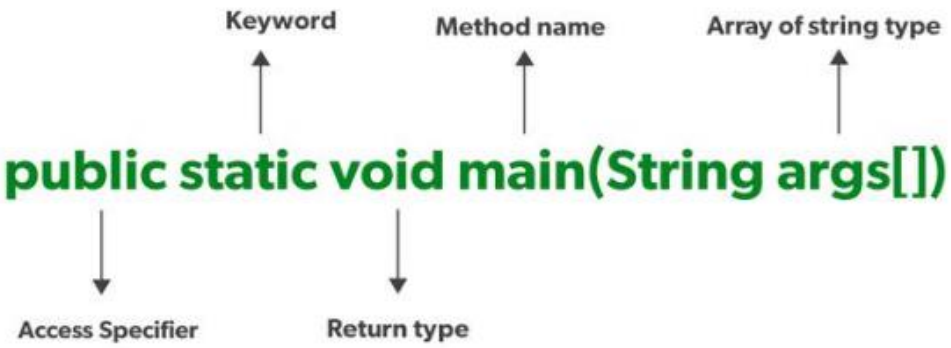
Java's main() method is the starting point from where the **JVM** starts the execution of a Java program. JVM will not execute the code, if the program is missing the main method. Hence, it is one of the most important methods of Java, and having a proper understanding of it is very important.

The Java compiler or JVM looks for the main method when it starts executing a Java program. The signature of the main method needs to be in a specific way for the JVM to recognize that method as its entry point. If we change the signature of the method, the program compiles but does not execute.

The execution of the Java program, the **java.exe** is called. The Java.exe in turn makes Java Native Interface or JNI calls, and they load the JVM. The java.exe parses the command line, generates a new String array, and invokes the main() method. By default, the main thread is always a non-daemon thread.

Syntax of main() Method

Syntax of the main() method is always written as:



Every word in the ***public static void main*** statement has a meaning in the JVM that is described below:

1. Public

It is an **Access modifier**, which specifies from where and who can access the method. Making the main() method public makes it globally available. It is made public so that JVM can invoke it from outside the class as it is not present in the current class.
If the main method is not public, its access is restricted.

2. Static

It is a **keyword** that is when associated with a method, making it a **class-related method**. The *main()* method is static so that JVM can invoke it without **instantiating the class**. This also saves the unnecessary wastage of memory which would have been used by the object declared only for calling the *main()* method by the JVM.
If you try to run Java code where main is not static, you will get an error.

3. Void

It is a **keyword** and is used to **specify that a method doesn't return anything**. As the *main()* method doesn't return anything, its return type is **void**. As soon as the *main()* method terminates, the Java program terminates too. Hence, it doesn't make any sense to return from the *main()* method as JVM can't do anything with its return value of it.
If main method is not void, we will get an error.

4. main

It is the **name of the Java main method**. It is the **identifier** that the JVM looks for as the **starting point of the Java program**. It's not a keyword.
If we change the name while initiating main method, we will get an error.

5. String[] args

It **stores Java command-line arguments** and is an array of type ***java.lang.String*** class. Here, the name of the String array is *args* but it is not fixed and the user can use any name in place of it.
Example: Execution Process of String[]

Overloading main() Method in Java

Overloading the main() method is possible in Java, meaning we can create any number of main() methods in a program.

To overload the main() method in Java, we need to create the main() method with different parameters.

Conclusion

Java main() method is the starting point of a Java program. It is the main body that is executed by the JVM, and without the main() method no Java program can be run without it.

Example 1:

```
class Test{
  public static void main(String args[]){

   System.out.println("Hello World.");
   }
}
```

Output

```
PS C:\Users\Admin\Desktop\Core Java pro> javac Test.java
PS C:\Users\Admin\Desktop\Core Java pro> java Test
Hello World.
PS C:\Users\Admin\Desktop\Core Java pro>
```

Example 2: We can place [] before args

```
class Test{
  public static void main(String []args){

   System.out.println("Hello World.");
   }
}
```

Output

```
PS C:\Users\Admin\Desktop\Core Java pro> javac Test.java
PS C:\Users\Admin\Desktop\Core Java pro> java Test
Hello World.
PS C:\Users\Admin\Desktop\Core Java pro>
```

Example 3: We can place [] just after String

```
class Test{
  public static void main(String[] args){

   System.out.println("Hello World.");
   }
}
```

Output:

```
PS C:\Users\Admin\Desktop\Core Java pro> javac Test.java
PS C:\Users\Admin\Desktop\Core Java pro> java Test
Hello World.
PS C:\Users\Admin\Desktop\Core Java pro>
```

Example 4: We can use three dots (...) instead of []

```
class Test{
  public static void main(String... args){

   System.out.println("Hello World.");
   }
}
```

Output

```
PS C:\Users\Admin\Desktop\Core Java pro> javac Test.java
PS C:\Users\Admin\Desktop\Core Java pro> java Test
Hello World.
PS C:\Users\Admin\Desktop\Core Java pro>
```

Example 5: We can use any name instead of args. Here I have used kumar

```
class Test{
  public static void main(String... kumar){

   System.out.println("Hello World.");
   }
}
```

Output:

```
PS C:\Users\Admin\Desktop\Core Java pro> javac Test.java
PS C:\Users\Admin\Desktop\Core Java pro> java Test
Hello World.
PS C:\Users\Admin\Desktop\Core Java pro>
```

Example 6 : We can exchange the place of public and static like below.

```
class Test{
   static public void main(String... kumar){

   System.out.println("Hello World.");
   }
}
```

Output

```
PS C:\Users\Admin\Desktop\Core Java pro> javac Test.java
PS C:\Users\Admin\Desktop\Core Java pro> java Test
Hello World.
PS C:\Users\Admin\Desktop\Core Java pro>
```

Example 7: If we have two class 1st class name is A And Second class name is Test. See the below example:

```java
public class A {

  public void show(){
    System.out.println("I m  in class A..");
  }
}
```

```java
class Test{
   static public void main(String... kumar){

      A ao=new A();
      ao.show();
   }
}
```

Ouput:

```
PS C:\Users\Admin\Desktop\Core Java pro> javac Test.java
PS C:\Users\Admin\Desktop\Core Java pro> java Test
I m  in class A..
PS C:\Users\Admin\Desktop\Core Java pro>
```

Example 8: If method is static, the we don't need to create object . See the below example:

```
public class A {

  public static void show(){
    System.out.println("I m  in class A..");
  }
}
```

```
class Test{
   static public void main(String... kumar){


      A.show();
   }
}
```

Output:

```
  PS C:\Users\Admin\Desktop\Core Java pro> javac Test.java
  PS C:\Users\Admin\Desktop\Core Java pro> java Test
  I m  in class A..
  PS C:\Users\Admin\Desktop\Core Java pro>
```

Example 9: In below example we are passing value at run time.

```
class Test{
   static public void main(String args[]){

    System.out.println(args[0]);
    System.out.println(args[1]);
    System.out.println(args[2]);


   }
}
```

Output: Please focus on execution comman.

```
PS C:\Users\Admin\Desktop\Core Java pro> javac Test.java
PS C:\Users\Admin\Desktop\Core Java pro> java Test Kumar nishant Mumbai
Kumar
nishant
Mumbai
PS C:\Users\Admin\Desktop\Core Java pro>
```

Example 10: If we try to run any class (in which there is no main method then it will show run time error but not at compile time)

```
public class A {

  public void show(){
    System.out.println("I m  in class A..");
  }
}
```

Output

```
PS C:\Users\Admin\Desktop\Core Java pro> javac A.java
PS C:\Users\Admin\Desktop\Core Java pro> java A
Error: Main method not found in class A, please define the main method as:
   public static void main(String[] args)
or a JavaFX application class must extend javafx.application.Application
PS C:\Users\Admin\Desktop\Core Java pro>
```